
Vault - testy

Opis testów i metodologii

Jan Kubów, Tomasz Gadziński, Karol Wołkowski

2026-06-24

Spis treści

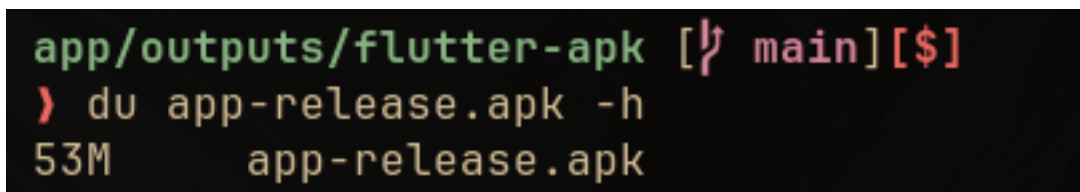
1	Testowanie widgetu	2
2	Testowanie rozmiaru aplikacji	2
2.1	Test ogólny	2
2.2	Test szczegółowy	3

1 Testowanie widgetu

- **Widget:** SingleAssetPageViewer
- **Testowane zachowanie i wygląd:**
 - Test ukrytego panelu *details* na start
 - Test pojawiającego się panelu *details* na interakcję użytkownika typu *drag up*
 - Test poprawnie wyświetlających się metadanych w panelu *details*
- **Mockowane dane:**
 - Nadpisana klasa **Asset**, z przestonowaną metodą `buildImage`, która zwraca zdjęcie statyczne z assetów aplikacji
 - Mock metadanych dla zdjęcia

2 Testowanie rozmiaru aplikacji

2.1 Test ogólny



```
app/outputs/flutter-apk [↵ main][$]  
> du app-release.apk -h  
53M      app-release.apk
```

Rysunek 1: Rys. 1: Rozmiar aplikacji po zbudowaniu release build

Zbudowana wersja release w paczce `.apk` dla androida waży 53MB. Nie jest to jednak liczba w 100% reprezentatywna, ponieważ upload do usługi *google play store*, optymalizuje build, rozbijając paczkę na części pierwsze i dostosowuje download size do każdego urządzenia.

Natomiast powyższa paczka jest przeznaczona na wszystkie architektury. Po rozdzieleniu budowania dla każdej architektury osobno, rozmiary prezentują się następująco:

Architektura	Rozmiar
arm	17MB
arm64	19.4MB

Architektura	Rozmiar
x86	20.9MB

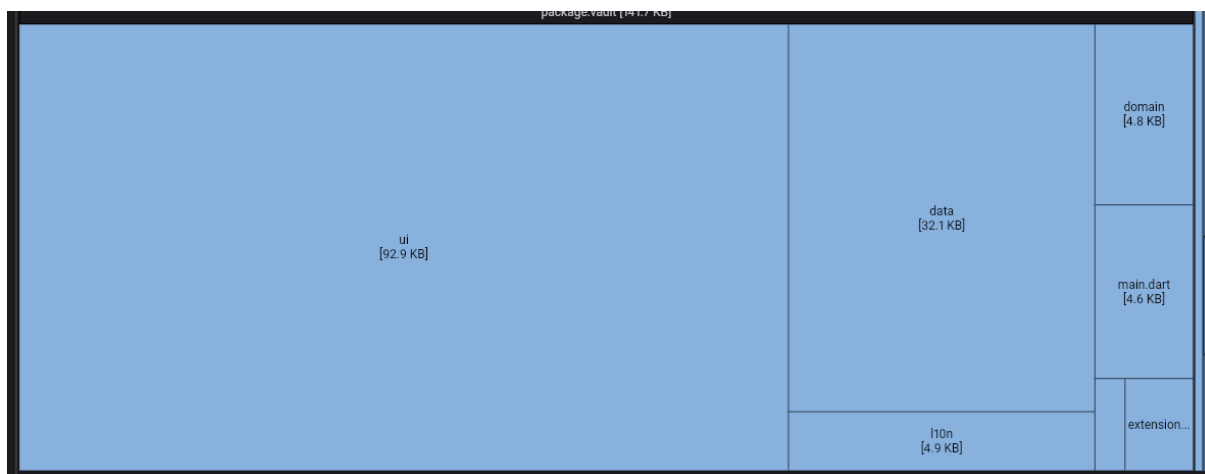
2.2 Test szczegółowy

Note

Test szczegółowy z rozbiciem na paczki i składowe, został przeprowadzony za pomocą narzędzia dostarczonego przez *flutter framework* - **size analysis tool**. Do analizy wykorzystany został build na najpopularniejszą architekturę procesora na rynku czyli arm64

Library or Class	Size	% of Total Size
▼ Root	18.6 MB	100.00 %
▼ lib	17.4 MB	93.86 %
▼ arm64-v8a	17.2 MB	92.85 %
libflutter.so (Flutter Engine)	11.0 MB	59.45 %
▼ libapp.so (Dart AOT)	6.1 MB	32.72 %
> package:flutter	2.9 MB	15.62 %
> @unknown	575.5 KB	3.03 %
> dartcore	287.7 KB	1.51 %
> dart.mixin.deduplication	283.1 KB	1.49 %
> dartui	225.2 KB	1.18 %
> darttyped_data	172.3 KB	0.91 %
> package:flutter_localizations	154.7 KB	0.81 %
> @stubs	152.2 KB	0.80 %
> package:vault	143.9 KB	0.76 %
> @shared	139.3 KB	0.73 %
> dartio	119.3 KB	0.63 %
> dart_http	104.9 KB	0.55 %
> package:intl	102.2 KB	0.54 %
> dart:async	98.7 KB	0.52 %
> dart:collection	78.2 KB	0.41 %
> dart_internal	72.0 KB	0.38 %
> package:material_color_utilities	70.6 KB	0.37 %
> dart_compact_hash	66.1 KB	0.35 %
> dart:convert	61.0 KB	0.32 %
> package:unicode_extensions	42.6 KB	0.22 %

Rysunek 2: Rys. 2: Drzewo struktury projektów przedstawiające rozmiary paczek



Rysunek 3: Rys. 3: Treemap przedstawiający rozkład rozmiaru dla kodu napisanego przez nas

Jak widać na rysunku 2, największa ilość pamięci zabiera sam framework, oraz paczki darta wspomagające framework. Paczki dodatkowe które instalowaliśmy w ramach developmentu, nie są większe od naszego kodu. Widzimy że zależności nie tłoczą niepotrzebnie pamięci

Z rysunku 3, możemy wywnioskować, że największą ilość pamięci zabiera kod do samego UI